Listy Dwukierunkowe

Generated by Doxygen 1.18.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 Iterator Class Reference . . . 5

3.1.1 Detailed Description . . . 5

3.1.2 Constructor & Destructor Documentation . . . 6

3.1.2.1 Iterator() . . . 6

3.1.3 Member Function Documentation . . . 6

3.1.3.1 getValue() . . . 6

3.1.3.2 hasNext() . . . 6

3.1.3.3 hasPrevious() . . . 6

3.1.3.4 next() . . . 7

3.1.3.5 previous() . . . 7

3.1.3.6 setCurrent() . . . 7

3.2 List Class Reference . . . 7

3.2.1 Detailed Description . . . 8

3.2.2 Member Function Documentation . . . 8

3.2.2.1 addAt() . . . 8

3.2.2.2 addFirst() . . . 9

3.2.2.3 addLast() . . . 9

3.2.2.4 isEmpty() . . . 9

3.2.2.5 removeAt() . . . 10

3.2.2.6 removeFirst() . . . 10

3.2.2.7 removeLast() . . . 10

3.3 Node Class Reference . . . 10

3.3.1 Detailed Description . . . 11

3.3.2 Constructor & Destructor Documentation . . . 11

3.3.2.1 Node() . . . 11

3.4 NodeFactory Class Reference . . . 11

3.4.1 Detailed Description . . . 12

3.4.2 Member Function Documentation . . . 12

3.4.2.1 create() . . . 12

4 File Documentation 13

4.1 Iterator.cpp File Reference . . . 13

4.1.1 Detailed Description . . . 13

4.2 Iterator.hpp File Reference . . . 13

4.2.1 Detailed Description . . . 14

4.3 Iterator.hpp . . . 14

4.4 List.cpp File Reference . . . 14
4.4.1 Detailed Description . . . 14
4.5 List.hpp File Reference . . . 15
4.5.1 Detailed Description . . . 15
4.6 List.hpp . . . 15
4.7 main.cpp File Reference . . . 16
4.7.1 Detailed Description . . . 16
4.7.2 Function Documentation . . . 16
4.7.2.1 main() . . . 16
4.8 Node.cpp File Reference . . . 17
4.8.1 Detailed Description . . . 17
4.9 Node.hpp File Reference . . . 17
4.9.1 Detailed Description . . . 17
4.10 Node.hpp . . . 18
4.11 NodeFactory.cpp File Reference . . . 18
4.11.1 Detailed Description . . . 18
4.12 NodeFactory.hpp File Reference . . . 18
4.12.1 Detailed Description . . . 19
4.13 NodeFactory.hpp . . . 19

Index **21**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Iterator
: Allows moving through the list 5

List
: Implements a doubly linked list 7

Node
: Represents one element of a doubly linked list 10

NodeFactory
: Creates new nodes for the list 11

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

Iterator.cpp
Implementation of the Iterator class . . . 13
Iterator.hpp
Declaration of the Iterator class . . . 13
List.cpp
Implementation of the List class . . . 14
List.hpp
Declaration of the List class . . . 15
main.cpp
Main function for testing the doubly linked list . . . 16
Node.cpp
Implementation of the Node class . . . 17
Node.hpp
Declaration of the Node class . . . 17
NodeFactory.cpp
Implementation of the NodeFactory class . . . 18
NodeFactory.hpp
Declaration of the NodeFactory class . . . 18

Chapter 3

Class Documentation

3.1 Iterator Class Reference

Allows moving through the list.

`#include <Iterator.hpp>`

Public Member Functions

- Iterator ()
  Creates an empty iterator.
- void setCurrent (Node *node)
  Sets the current node.
- int getValue ()
  Returns the value of the current node.
- bool hasNext ()
  Checks if the next node exists.
- bool hasPrevious ()
  Checks if the previous node exists.
- int next ()
  Moves to the next node.
- int previous ()
  Moves to the previous node.

3.1.1 Detailed Description

Allows moving through the list.

The iterator can move forward and backward and read the value of the current node.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Iterator()

```
Iterator::Iterator ()
```

Creates an empty iterator.

Creates an iterator with no current node.

3.1.3 Member Function Documentation

3.1.3.1 getValue()

```
int Iterator::getValue ()
```

Returns the value of the current node.

Returns

Value stored in the current node.

Node value or -1 if the current node does not exist.

3.1.3.2 hasNext()

```
bool Iterator::hasNext ()
```

Checks if the next node exists.

Returns

True if the next node exists, otherwise false.

3.1.3.3 hasPrevious()

```
bool Iterator::hasPrevious ()
```

Checks if the previous node exists.

Returns

True if the previous node exists, otherwise false.

3.1.3.4 next()

```
int Iterator::next ()
```

Moves to the next node.

Moves the iterator to the next node.

Returns

Value of the next node.

Value of the next node or -1 if it does not exist.

3.1.3.5 previous()

```
int Iterator::previous ()
```

Moves to the previous node.

Moves the iterator to the previous node.

Returns

Value of the previous node.

Value of the previous node or -1 if it does not exist.

3.1.3.6 setCurrent()

```
void Iterator::setCurrent (
            Node * node)
```

Sets the current node.

Parameters

node	Pointer to the node.

The documentation for this class was generated from the following files:

- Iterator.hpp
- Iterator.cpp

3.2 List Class Reference

Implements a doubly linked list.

```
#include <List.hpp>
```

Public Member Functions

- **List** ()
  Creates an empty list.
- ~**List** ()
  Deletes the list and frees its memory.
- bool isEmpty ()
  Checks if the list is empty.
- void addFirst (int value)
  Adds a new element at the beginning.
- void **showAll** ()
  Displays all elements from first to last.
- void addLast (int value)
  Adds a new element at the end.
- void addAt (int index, int value)
  Adds an element at the given index.
- void removeFirst ()
  Removes the first element.
- void removeLast ()
  Removes the last element.
- void removeAt (int index)
  Removes an element at the given index.
- void **showReverse** ()
  Displays all elements from last to first.
- void **showNext** ()
  Moves the iterator forward and displays the next value.
- void **showPrevious** ()
  Moves the iterator backward and displays the previous value.
- void **clear** ()
  Removes all elements from the list.

3.2.1 Detailed Description

Implements a doubly linked list.

The list allows adding, removing and displaying elements. It also supports moving through the list using an iterator.

3.2.2 Member Function Documentation

3.2.2.1 addAt()

```
void List::addAt (
            int index,
            int value)
```

Adds an element at the given index.

Parameters

index	Position where the element will be added.
value	Value to add.

3.2.2.2 addFirst()

```
void List::addFirst (
            int value)
```

Adds a new element at the beginning.

Adds a new element at the beginning of the list.

Parameters

value	Value to add.

3.2.2.3 addLast()

```
void List::addLast (
            int value)
```

Adds a new element at the end.

Adds a new element at the end of the list.

Parameters

value	Value to add.

3.2.2.4 isEmpty()

```
bool List::isEmpty ()
```

Checks if the list is empty.

Returns

True if the list is empty, otherwise false.

3.2.2.5 removeAt()

```
void List::removeAt (
            int index)
```

Removes an element at the given index.

Parameters

index	Position of the element to remove.

3.2.2.6 removeFirst()

```
void List::removeFirst ()
```

Removes the first element.

Removes the first element of the list.

3.2.2.7 removeLast()

```
void List::removeLast ()
```

Removes the last element.

Removes the last element of the list.

The documentation for this class was generated from the following files:

- List.hpp
- List.cpp

3.3 Node Class Reference

Represents one element of a doubly linked list.

```
#include <Node.hpp>
```

Public Member Functions

- Node (int number)

  Creates a new node with the given value.

Public Attributes

- int **data**

  Value stored in the node.
- Node * **next**

  Pointer to the next node.
- Node * **previous**

  Pointer to the previous node.

3.3.1 Detailed Description

Represents one element of a doubly linked list.

Each node stores a value and pointers to the next and previous nodes.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 Node()

```
Node::Node (
            int number)
```

Creates a new node with the given value.

Parameters

number	Value to store in the node.
number	Value to store in the node.

The next and previous pointers are set to nullptr.

The documentation for this class was generated from the following files:

- Node.hpp
- Node.cpp

3.4 NodeFactory Class Reference

Creates new nodes for the list.

```
#include <NodeFactory.hpp>
```

Static Public Member Functions

- static Node * create (int value)

  Creates a new node with the given value.

3.4.1 Detailed Description

Creates new nodes for the list.

3.4.2 Member Function Documentation

3.4.2.1 create()

```
Node * NodeFactory::create (
            int value)  [static]
```

Creates a new node with the given value.

Creates a new node.

Parameters

value	Value to store in the node.

Returns

Pointer to the created node.

Parameters

value	Value to store in the node.

Returns

Pointer to the new node.

The documentation for this class was generated from the following files:

- NodeFactory.hpp
- NodeFactory.cpp

Chapter 4

File Documentation

4.1 Iterator.cpp File Reference

Implementation of the Iterator class.

```
#include "Iterator.hpp"
#include "Node.hpp"
```

4.1.1 Detailed Description

Implementation of the Iterator class.

Author

Piotr Majchrzak

Date

18/10/2026

4.2 Iterator.hpp File Reference

Declaration of the Iterator class.

```
#include <stdio.h>
#include "Node.hpp"
```

Classes

- class Iterator

  Allows moving through the list.

4.2.1 Detailed Description

Declaration of the Iterator class.

Author

Piotr Majchrzak

Date

18/10/2026

4.3 Iterator.hpp

Go to the documentation of this file.

```
00001
00007
00008 #ifndef Iterator_hpp
00009 #define Iterator_hpp
00010
00011 #include <stdio.h>
00012 #include "Node.hpp"
00013
00021 class Iterator {
00022 private:
00024     Node *current;
00025
00026 public:
00030     Iterator();
00031
00036     void setCurrent(Node *node);
00037
00042     int getValue();
00043
00048     bool hasNext();
00049
00054     bool hasPrevious();
00055
00060     int next();
00061
00066     int previous();
00067 };
00068
00069 #endif /* Iterator_hpp */
```

4.4 List.cpp File Reference

Implementation of the List class.

```
#include "List.hpp"
#include "Node.hpp"
#include "NodeFactory.hpp"
#include <iostream>
```

4.4.1 Detailed Description

Implementation of the List class.

Author

Piotr Majchrzak

Date

18/10/2026

4.5 List.hpp File Reference

Declaration of the List class.

```
#include <stdio.h>
#include "Node.hpp"
#include "Iterator.hpp"
```

Classes

- class List

  Implements a doubly linked list.

4.5.1 Detailed Description

Declaration of the List class.

Author

Piotr Majchrzak

Date

18/10/2026

4.6 List.hpp

Go to the documentation of this file.

```
00001
00007
00008 #ifndef List_hpp
00009 #define List_hpp
00010
00011 #include <stdio.h>
00012 #include "Node.hpp"
00013 #include "Iterator.hpp"
00014
00022 class List {
00023 private:
00025     Node *first;
00026
00028     Node *last;
00029
00031     int size;
00032
00034     Iterator iterator;
00035
00036 public:
00040     List();
00041
00045     ~List();
00046
00051     bool isEmpty();
00052
00057     void addFirst(int value);
00058
00062     void showAll();
00063
00068     void addLast(int value);
00069
```

```
00075     void addAt(int index, int value);
00076
00080     void removeFirst();
00081
00085     void removeLast();
00086
00091     void removeAt(int index);
00092
00096     void showReverse();
00097
00101     void showNext();
00102
00106     void showPrevious();
00107
00111     void clear();
00112 };
00113
00114 #endif /* List_hpp */
```

4.7 main.cpp File Reference

Main function for testing the doubly linked list.

```
#include <iostream>
#include "List.hpp"
```

Functions

- int main (int argc, const char *argv[])

  Runs tests for the List class.

4.7.1 Detailed Description

Main function for testing the doubly linked list.

Author

Piotr Majchrzak

4.7.2 Function Documentation

4.7.2.1 main()

```
int main (
            int argc,
            const char * argv[])
```

Runs tests for the List class.

Parameters

argc	Number of command line arguments.

argv	Array of command line arguments.

Returns

0 when the program finishes successfully.

4.8 Node.cpp File Reference

Implementation of the Node class.

```
#include "Node.hpp"
```

4.8.1 Detailed Description

Implementation of the Node class.

Author

Piotr Majchrzak

Date

18/10/2026

4.9 Node.hpp File Reference

Declaration of the Node class.

```
#include <stdio.h>
```

Classes

- class Node

  Represents one element of a doubly linked list.

4.9.1 Detailed Description

Declaration of the Node class.

Author

Piotr Majchrzak

Date

18/10/2026

4.10 Node.hpp

Go to the documentation of this file.

```
00001
00007
00008 #ifndef Node_hpp
00009 #define Node_hpp
00010
00011 #include <stdio.h>
00012
00019 class Node {
00020 public:
00022     int data;
00023
00025     Node *next;
00026
00028     Node *previous;
00029
00034     Node(int number);
00035 };
00036
00037 #endif /* Node_hpp */
```

4.11 NodeFactory.cpp File Reference

Implementation of the NodeFactory class.

```
#include "NodeFactory.hpp"
```

4.11.1 Detailed Description

Implementation of the NodeFactory class.

Author

Piotr Majchrzak

Date

18/10/2026

4.12 NodeFactory.hpp File Reference

Declaration of the NodeFactory class.

```
#include "Node.hpp"
#include <stdio.h>
```

Classes

- class NodeFactory

  Creates new nodes for the list.

4.12.1 Detailed Description

Declaration of the NodeFactory class.

Author

Piotr Majchrzak

Date

18/10/2026

4.13 NodeFactory.hpp

Go to the documentation of this file.

```
00001
00007
00008 #ifndef NodeFactory_hpp
00009 #define NodeFactory_hpp
00010
00011 #include "Node.hpp"
00012 #include <stdio.h>
00013
00018 class NodeFactory {
00019 public:
00025     static Node *create(int value);
00026 };
00027
00028 #endif /* NodeFactory_hpp */
```

Index

addAt
    List, 8
addFirst
    List, 9
addLast
    List, 9

create
    NodeFactory, 12

getValue
    Iterator, 6

hasNext
    Iterator, 6
hasPrevious
    Iterator, 6

isEmpty
    List, 9
Iterator, 5
    getValue, 6
    hasNext, 6
    hasPrevious, 6
    Iterator, 6
    next, 6
    previous, 7
    setCurrent, 7
Iterator.cpp, 13
Iterator.hpp, 13

List, 7
    addAt, 8
    addFirst, 9
    addLast, 9
    isEmpty, 9
    removeAt, 9
    removeFirst, 10
    removeLast, 10
List.cpp, 14
List.hpp, 15

main
    main.cpp, 16
main.cpp, 16
    main, 16

next
    Iterator, 6
Node, 10
    Node, 11
Node.cpp, 17
Node.hpp, 17
NodeFactory, 11
    create, 12
NodeFactory.cpp, 18
NodeFactory.hpp, 18

previous
    Iterator, 7

removeAt
    List, 9
removeFirst
    List, 10
removeLast
    List, 10

setCurrent
    Iterator, 7